

AtlasAutoware: Taking a 1/10-Scale Autonomous Race Car from Simulation to Hardware

Eshan Iyer

September 2, 2026

Introduction

This project picks up the 1/10-scale self-driving car in the SysLab annex where the 2024 Senior Research work left it [10]. That report ended with a car that could follow a hallway wall using a hobby ESC, a PWM breakout board and the DonkeyCar library, and it recommended that the next group not deviate from the F1TENTH bill of materials, because every substitution costs weeks. I took that advice in one direction and ignored it in another: I rebuilt the actuation around the VESC motor controller that the previous report gave up on, so that the car speaks the same interfaces as the F1TENTH reference platform [1, 8], but I kept the sensors the lab actually owns (a Slamtec RPLIDAR C1 and an Orbbec Gemini 335) rather than the Hokuyo and OAK-D the reference design assumes.

The goal was a car that runs our team’s simulation racing stack, AtlasAutoware [3, 4], on real hardware with the fewest possible changes to the software. The stack was written against the F1TENTH gym simulator, and the plan was to keep the planner and controller untouched and only swap the sensor and actuator layers. I divided the work into three stages:

1. Actuation: make the VESC drive the motor and steering reliably from the Jetson, in closed loop, with usable odometry.
2. Sensing: bring the lidar and the camera into ROS 2 in the exact shape the racing stack expects, including the camera’s inertial measurement unit.
3. Perception: train and deploy a detector for other F1TENTH cars so the head-to-head agent has a second source of opponent positions.

All three stages were completed and are checked into the team repository, together with a `hardware/` folder that lets someone rebuild the car’s configuration from GitHub. What has not been done is validate the autonomous controller on the floor with the new drivetrain; the competition that was the deadline for that (Boston, September 6–7, 2026) was cancelled when most of the team withdrew. The Results section separates what was measured from what still needs a track.

Prior Approaches

Three earlier designs shaped the choices here.

The 2024 report [10] drove the same chassis through a PCA9685 PWM generator into an 80 A hobby ESC. That path works with DonkeyCar and needs no motor controller protocol, but it is open loop: the ESC reports nothing back, so there is no wheel speed, no current, no temperature, and no way to calibrate throttle except by driving. The report describes spending several months trying to control a VESC over USB and through an Arduino before switching to the hobby ESC.

The F1TENTH reference stack [8] uses the VESC over USB with a ROS 2 driver, converts Ackermann commands into VESC RPM set-points, and reconstructs odometry from the VESC’s electrical RPM. It expects a Hokuyo lidar on Ethernet and a specific gamepad. The Jetson in the lab already had this stack installed, but with a lidar launch file for a sensor the car does not have and a control mode that had caused a motor to overheat.

The team’s own AtlasAutoware stack [4] contributes the planning and control layer: a linear time-varying model predictive controller for raceline tracking with a persistent warm-started quadratic program (0.6 ms mean solve time in the reported benchmark), a Model- and Acceleration-based Pursuit fallback, minimum-curvature raceline refinement, a friction-limited velocity profile, an IMU-based traction governor, actuation-latency compensation and a Frenet-frame overtaking planner. Those results were obtained in the simulator against a kinematic bicycle plant. The hardware layer in that paper assumed an OAK-D Pro camera; ours is an Orbbec.

MuSHR [2] is the other common open racecar design and uses the same VESC interface, which is one reason to adopt it: any VESC-based stack becomes usable on this car.

System Architecture

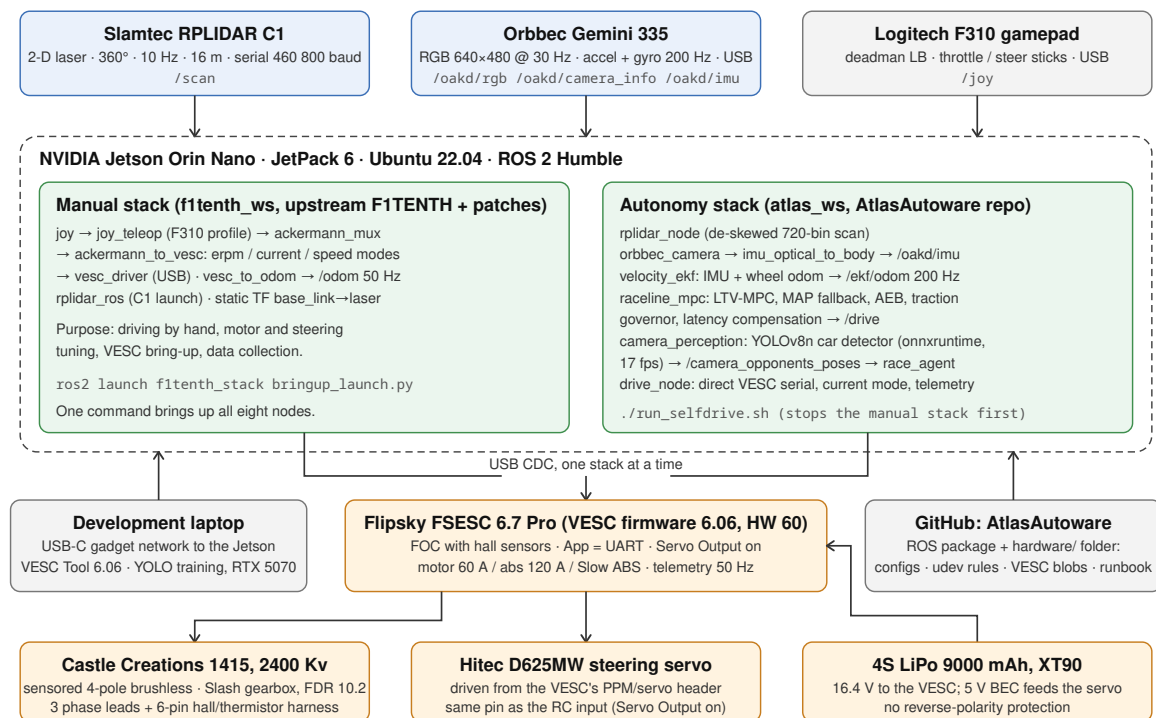


Figure 1: The car as built. Blue: sensors. Green: the two ROS 2 workspaces on the Jetson. Amber: the power and actuation chain. Grey: the development laptop and the repository. Only one workspace talks to the VESC at a time; the autonomy launcher stops the manual stack before it starts.

Figure 1 shows the system. Two ROS 2 workspaces live on the Jetson and share the same hardware. The manual stack is the upstream F1TENTH system with our patches: a gamepad drives the car through the F1TENTH Ackermann mux and VESC driver, and it is what I use to tune the motor controller and to drive the car by hand. The autonomy stack is the AtlasAutoware package. It has its own lidar node (which de-skews each scan using the velocity estimate), its own VESC driver, the velocity EKF, the MPC racing node and the camera perception node. The two stacks cannot both own the VESC’s USB port, so the autonomy launcher kills the manual stack before starting.

Table 1 lists the hardware. Compared with the 2024 build, the ESC, motor, camera, lidar and battery are all different; the chassis, Jetson and servo are the same.

Part	Notes
NVIDIA Jetson Orin Nano	JetPack 6 (L4T R36.5), Ubuntu 22.04, ROS 2 Humble. Reached from the laptop over the USB-C gadget network at 192.168.55.1.
Flipsky FSESC 6.7 Pro	VESC firmware 6.06 on the HW 60 profile. Replaced once after reversed battery leads destroyed the first board (the VESC has no reverse-polarity protection).
Castle Creations 1415, 2400 Kv	Sensored 4-pole brushless motor, 5 mm shaft, with the hall/thermistor harness wired to the VESC’s 6-pin sensor port. Replaced the stock Traxxas Velineon 3500.
Traxxas Slash drivetrain	Final drive ratio 10.2, 0.109 m tires, on a laser-cut deck.
Hitec D625MW	Steering servo on the VESC’s servo header.
Slamtec RPLIDAR C1	2-D lidar, 360°, 10 Hz, 16 m, 460 800 baud over a CP210x serial adapter [15].
Orbbec Gemini 335	RGB-D camera with a 200 Hz accelerometer and gyroscope [12]; used here for colour and the IMU, depth off.
Logitech F310	Wired gamepad, XInput mode.
4S LiPo, 9000 mAh, XT90	16.4 V measured. The VESC’s 5 V BEC powers the servo.

Table 1: Hardware on the car as of September 2026.

Stage 1: Actuation

Getting the VESC to drive the motor

The VESC is attractive because it closes the loop: it measures phase currents, estimates rotor speed, exposes a PID speed controller, and reports voltage, current and temperature over USB at 50 Hz. It is also unforgiving. Three problems consumed most of this stage.

The first was motor control itself. With the stock sensorless Velineon motor, the VESC’s field-oriented control depends on a back-EMF observer that has nothing to observe at standstill. Below roughly 2000 electrical RPM the motor cogged, drew current without moving, and twice heated its phase wires until solder joints failed. Tuning helped (the working configuration used an observer gain of 62, a sensorless threshold of 2000 electrical RPM, motor current limits of 50–60 A and the firmware’s slow absolute-current limit, without which current ripple tripped the over-current fault), but the underlying dead zone was physical. I replaced the motor with a sensored Castle 1415 whose hall sensors give the controller rotor position from zero speed. Detection with hall sensors produced a hall table and motor parameters that the firmware’s observer tracked cleanly up to the 10 000 electrical RPM I tested on a stand.

The second was the software’s response to the same dead zone. Both stacks originally commanded the VESC in RPM mode. Someone on the team had already added a custom `erpm` control mode to the F1TENTH `ackermann_to_vesc` node that never commands below 3000 electrical RPM, and a matching `current` mode in the AtlasAutoware `drive_node` that maps the planner’s speed to motor current with a feed-forward minimum of 10 A, so the car breaks stiction the instant a non-zero speed is requested instead of waiting for the RPM loop to wind up. With a sensored motor these are no longer necessary for safety, but current mode remains the default for the autonomy path because it gives the MPC direct torque authority. The speed-to-RPM gain that odometry

depends on was also wrong in both stacks (it was the reference car’s value, 4614); for this drivetrain it is $60 \cdot 10.2 \cdot 2 / (\pi \cdot 0.109) \approx 3575$ electrical RPM per m/s.

The third problem is the one worth recording for whoever comes next, because it cost a day and had no error message. After the replacement VESC was installed and configured, the motor ran from VESC Tool on the laptop but not from ROS: every command from the Jetson produced zero duty, zero current and fault code zero. Rather than keep guessing from the ROS side, I wrote a small Python implementation of the VESC serial protocol (CRC-16/XMODEM packet framing, `COMM_GET_VALUES`, `COMM_SET_RPM`, `COMM_GET_MCCONF`, `COMM_GET_APPCONF`) and a decoder for the firmware’s serialized configuration, taking the field order and the float16 scale factors from `confgenerator.c` in the 6.06 firmware source [16]. That made the cause visible in one read. A fresh Flip-sky board ships with the RC-input application enabled (`app_to_use = PPM_UART`) and servo output disabled. On this hardware the PPM input and the servo output share one pin, so the steering servo’s signal wire was being decoded as an RC receiver, and the PPM application overwrote every USB command with “neutral” fifty times a second. Setting `servo_out_enable`, switching the application to UART and rebooting the controller fixed both the motor and the steering at once. The same tooling then wrote the current limits (60 A motor, 120 A absolute, slow absolute-current limit on) and saved the complete motor and application configurations as binary blobs in the repository, so the next dead board can be restored in seconds without VESC Tool.

What the actuation layer now provides

From the manual stack, the F310 gamepad drives the car with a dead-man button. From the autonomy stack, `drive_node` auto-detects the VESC on its stable udev path, commands current or speed, publishes wheel-speed odometry at 20 Hz and logs battery voltage, MOSFET temperature and fault codes. On the bench the VESC held commanded speeds of 1500, 3500, 6000 and 9000 electrical RPM to within 1% at 1.5–3 A.

Stage 2: Sensing

Lidar

The lidar is an RPLIDAR C1, not the A2 the launch files assumed. The C1 uses 460 800 baud, and the A2, A3 and S1 launch files all time out on it silently. Both stacks were changed to the C1 settings; the F1TENTH bring-up now starts the lidar itself instead of requiring a second launch. The scan arrives at 10 Hz with 16 m range.

Camera and IMU

The AtlasAutoware stack was written for an OAK-D Pro and consumes three topics from it: a colour image, its calibration, and a single `sensor_msgs/Imu` message carrying both acceleration and angular rate. The important consumer is not vision. The `velocity_ekf` node fuses the IMU with wheel odometry into a 200 Hz velocity estimate that the lidar node uses to de-skew each 100 ms sweep, and the MPC’s traction governor reads the gyro yaw rate to back off speed when the car slides. Without an IMU both features are inert.

I replaced the OAK-D node with the official Orbbec ROS 2 driver [13] (OrbbecSDK v2 branch, built on the Jetson) and remapped its colour topics onto the names the stack

expects. The IMU needed more than a rename. The Orbbec reports acceleration and angular rate in the camera’s optical frame (x right, y down, z forward), while the EKF and the governor assume ROS body axes (x forward, z up) [9]. Fed directly, the EKF would have integrated lateral acceleration as forward acceleration and treated roll rate as yaw rate. A 60-line relay node, `imu_optical_to_body`, rotates both vectors and their covariances. It also publishes with reliable QoS, because the EKF subscribes with the ROS 2 default and a best-effort publisher never matches it: the first version of the relay produced a 200 Hz stream that nobody received, with no warning beyond one line in the log.

Building the driver exposed a JetPack quirk that will bite any C++ package using OpenCV: the image had NVIDIA’s OpenCV 4.8 development headers installed alongside Ubuntu’s 4.5.4 libraries, so every `find_package(OpenCV)` failed. Downgrading the development package to 4.5.4 to match the runtime fixed it. The camera negotiates USB 2.0 with the cable currently on the car, which is enough for 640×480 colour at 30 Hz and the IMU at 200 Hz; depth would need the USB 3 cable.

With the car level and still, the rotated IMU reports gravity on body $+z$ at 9.75 m/s^2 , and `/ekf/odom` publishes at 200 Hz whenever the VESC odometry is present.

Stage 3: Perception

Why a car detector

The stack’s head-to-head agent, `race_agent`, fuses opponent positions from the lidar with any that arrive on `/camera_opponents_poses`. The camera perception node that publishes that topic was already written: it runs a single-class YOLOv8 detector, turns each box into a range from the box width and the camera’s focal length, and hands the result to the agent’s opponent tracker. What was missing was a model; the repository contained an empty dataset scaffold and a training script that had never been run.

Data

No labelled images of F1TENTH cars existed in the lab, and there was only one car to photograph. I assembled a dataset from four public Roboflow Universe collections [7, 6, 14, 5], all CC BY 4.0, with a merge script that maps each source’s class names (`car`, `Cars`, `f1tenth`, `racecar`) to one class, drops boxes for anything else, and keeps each source’s own train/validation split so that the augmented copies in one collection never leak across the split. The result was 4907 training and 547 validation images with 5389 boxes.

The first model trained on that data reached a mean average precision at IoU 0.5 of 0.974 on the held-out images, but it also called a hand or a face at the lens a car at close range in 21 of 185 test frames. None of the public images contain a person 30 cm from the camera. I therefore recorded 185 frames from the car’s own Orbbec while walking it around the hallway and waving hands, tools and a backpack in front of it, added them to the training set with empty label files, and fine-tuned for 20 epochs. False detections on those frames dropped to 0 of 185, and the validation metrics did not suffer (Table 2). These negatives stay off GitHub because they contain people; the merge and training scripts in `tools/` rebuild everything else from the public downloads.

Model	mAP ₅₀	mAP _{50:95}	Prec.	Rec.	False cars / 185 neg.
YOLOv8n, public data, 50 epochs	0.974	0.754	0.993	0.971	21
+ Orbbec negatives, 20-epoch fine-tune	0.983	0.752	0.989	0.976	0

Table 2: Detector results on the 547-image validation split (578 after adding a share of the negatives). Training on an RTX 5070 laptop GPU took 17 minutes for the first run and 9 for the fine-tune.

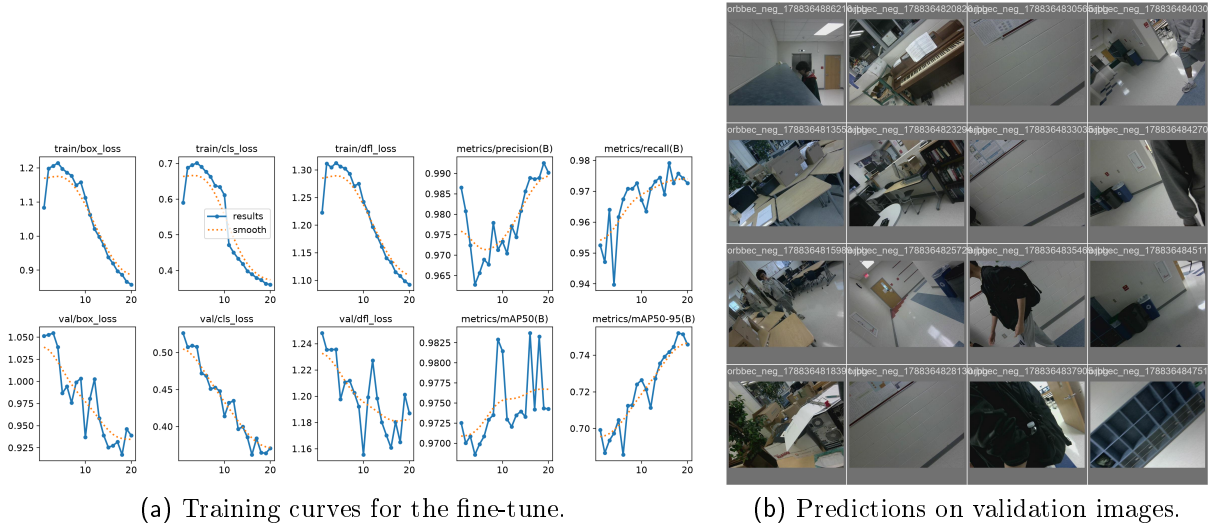


Figure 2: Detector training. Validation images are from the public collections.

Running it on the Jetson

The perception node offered three inference backends: a TensorRT engine, OpenCV’s CUDA DNN, and OpenCV on the CPU. None of them could run on this Jetson. The image has no CUDA toolkit or TensorRT installed, Ubuntu’s OpenCV has no CUDA support, and OpenCV 4.5.4’s DNN module fails on the YOLOv8 head’s reshape layer. I added a fourth backend on ONNX Runtime [11], which installs from a wheel and needs nothing from NVIDIA. Through that path the 416-pixel model runs at 58 ms per frame (17 Hz) and the 640-pixel model at 124 ms on the Orin’s CPU with four threads. On 40 real validation images the deployed 416 model found the car in every one.

Two guards were added in the node because a detector that is right 99% of the time is still dangerous when the 1% is a hand at the lens reported as an opponent at 0.2 m. Detections implying a range under 0.8 m or a box wider than 60% of the frame are dropped; inside that range the lidar owns the problem anyway. The confidence threshold was raised from 0.35 to 0.5. The node is off by default in the car launch and enabled with `use_perception:=true`, since it costs a CPU core and only the head-to-head agent uses it.

What Is Different About This System

Most of the individual pieces exist elsewhere; the combination and a few of the methods do not.

The planner never changed. The MPC, the velocity profiler, the raceline tools and the traction governor are the same code that runs in the simulator. Every hardware difference was absorbed in three places: the drive node’s backend, a camera relay, and configuration. This is the parity the AtlasAutoware paper argued for [4], and it held on real hardware.

Closed-loop actuation with direct torque authority. The 2024 build could only send a PWM pulse. This car reports wheel speed at 20 Hz, battery voltage and temperatures, and lets the controller command current rather than speed, which is what removed the launch stall.

The camera is a motion sensor first. The Orbbec’s main contribution to lap time is its IMU: it makes the lidar scans geometrically honest at speed and gives the governor a yaw rate to react to. The vision model is the second use, not the first.

Firmware-level diagnosis. Being able to read and write the VESC’s full configuration from the Jetson without VESC Tool turned a silent failure into a one-line diagnosis, and it makes the motor controller’s state part of the repository rather than something that lives in one laptop’s settings.

Negatives from the deployed camera. Public datasets tell a detector what a car looks like; only the target camera can tell it what the false alarms look like. Two minutes of recording eliminated all 21 false detections.

Reproducibility as a deliverable. The `hardware/` folder holds the F1TENTH configuration overrides, the patch to the VESC driver, the udev rules that give the gamepad, lidar, VESC and camera stable names and permissions, the VESC configuration blobs and the tools that restore them, and a runbook of the failures described above. The 2024 report’s main recommendation was to reduce the time a new group spends rediscovering the car; this folder is the concrete form of that.

Results

Everything in Table 3 was measured on the car on 2 September 2026 with the wheels off the ground or the car stationary.

Quantity	Measured
Lidar scan rate (<code>/scan</code>)	10.0 Hz, 720 bins, 16 m
Camera colour (<code>/oakd/rgb</code>)	30 Hz at 640×480; 18–28 Hz with the detector running
Camera IMU (<code>/oakd/imu</code>)	195–200 Hz; gravity on body $+z = 9.75 \text{ m/s}^2$
Velocity EKF (<code>/ekf/odom</code>)	200 Hz
VESC telemetry / wheel odometry	50 Hz / 20 Hz; 16.4 V, MOSFETs 26–29 °C, fault 0
Speed tracking on a stand	commanded 1500 / 3500 / 6000 / 9000 eRPM; measured 1472 / 3493 / 5986 / 9006
Gamepad path, manual stack	full throttle held 10 000 eRPM at 3–4 A; steering full range
Detector on the Jetson CPU	58 ms per frame at 416 px; 0 false detections in 55 s of hands and faces

Table 3: Hardware measurements.

What is not in that table matters as much. The autonomous controller has not driven a lap on this drivetrain. The steering servo’s centre and end-point values in the configuration are still the reference car’s; the wheelbase and the lidar and camera mounting offsets in the static transforms are estimates, not measurements; the speed-to-RPM gain has been computed but not checked by driving a known distance; and the detector has only ever seen other F1TENTH cars in photographs, so its range estimate, which assumes a 0.30 m car width, is unverified against a real opponent.

Recommendations for the Next Group

Drive a measured five metres in the manual stack and compare the odometry before trusting any autonomy. Calibrate the servo the way the 2024 report did (a range finder on the front wheel), then enter the centre and limits in both stacks’ configuration. Install the JetPack CUDA and TensorRT packages when the Jetson is on a network that is not a captive portal; the perception node already prefers a TensorRT engine if one is present, and the 640-pixel ONNX export is in the repository for that purpose. Keep the battery connector keyed or marked: the VESC dies instantly on reversed leads, and a replacement took a week to arrive. Finally, read `hardware/RUNBOOK.md` before touching the VESC’s application settings; the shared PPM/servo pin will look like a dead motor controller to anyone who does not know about it.

Acknowledgments

[To be completed: research teacher, lab, teammates who built the chassis and the simulation stack, and whoever soldered the phase leads at three in the morning.]

References

- [1] F1tenth. URL <https://f1tenth.org/>.
- [2] Mushr: The uw open racecar project. URL <https://prl-mushr.github.io>.

- [3] AtlasAutoware. Atlasautoware: autonomous racing stack for the 1/10-scale car. URL <https://github.com/AtlasAutoware/atlasautoware>. commit cf39baa, 2 September 2026.
- [4] AtlasAutoware team. Atlasautoware: An integrated autonomous racing stack for 1/10-scale vehicles with gpu-accelerated perception and component-wise closed-loop evaluation. Draft manuscript, docs/paper/PAPER.md in the AtlasAutoware repository, 2026.
- [5] CU Robotics. Detect fltenth dataset (v1), 2022. URL <https://universe.roboflow.com/cu-robotics-qtbgu/detect-fltenth>. Roboflow Universe, CC BY 4.0.
- [6] CURC Autonomous Vehicle Project. F1tenth car detection dataset (v6), 2023. URL <https://universe.roboflow.com/curc-autonomous-vehicle-project/fltenth-car-detection>. Roboflow Universe, CC BY 4.0.
- [7] F1Tenth Cars. F1 tenth dataset (v1, fltenthcars), 2022. URL <https://universe.roboflow.com/fltenth-cars/f1-tenth>. Roboflow Universe, CC BY 4.0.
- [8] F1TENTH Foundation. fltenth_system: Ros 2 drivers and bring-up for the fltenth car. URL https://github.com/fltenth/fltenth_system.
- [9] Tully Foote and Mike Purvis. Rep 103: Standard units of measure and coordinate conventions, 2010. URL <https://www.ros.org/repos/rep-0103.html>.
- [10] Manav Gagvani. Senior research report: Reinforcement learning for a 1/10-scale self-driving car. TJHSST Senior Research, Computer Systems Laboratory, 2024.
- [11] Microsoft. Onnx runtime. URL <https://onnxruntime.ai/>.
- [12] Orbbec. Gemini 335 stereo vision 3d camera, . URL <https://www.orbbec.com/products/stereo-vision-camera/gemini-335/>.
- [13] Orbbec. Orbbecsdk_ros2: Ros 2 wrapper for orbbec 3d cameras, . URL https://github.com/orbbec/OrbbecSDK_ROS2.
- [14] pepperpeople. f110 dataset (v1), 2024. URL <https://universe.roboflow.com/pepperpeople/f110>. Roboflow Universe, CC BY 4.0.
- [15] Slamtec. Rplidar c1 360° laser range scanner. URL <https://www.slamtec.com/en/C1>.
- [16] Benjamin Vedder. vedderb/bldec: Vesc firmware source (release_6_06). URL <https://github.com/vedderb/bldec>.